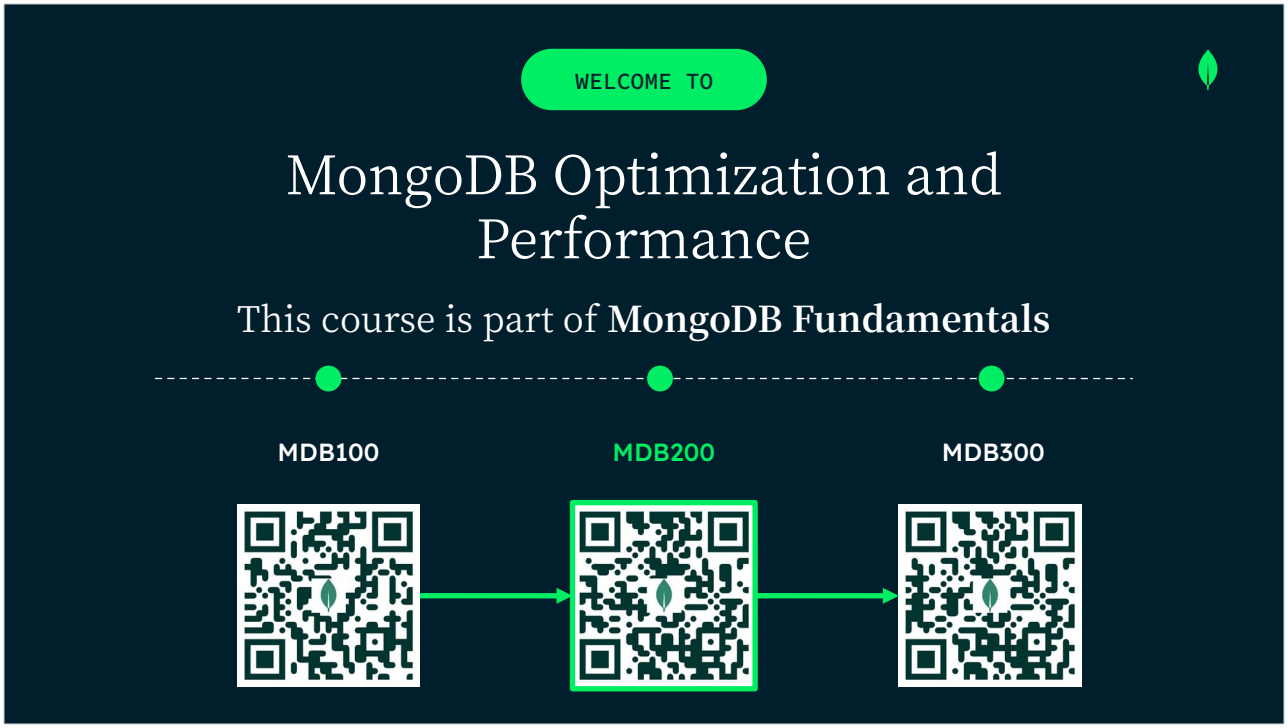
MDB200

Indexes and Optimization I

MongoDB Optimization and Performance

Release: 20260220



In MDB200, you will learn how to optimise database operations and improve performance.

MongoDB Fundamentals consists of three courses. We recommend that you do them in order before attempting 400-level, Specialty or Advanced courses.
https://mdb.link/mdb100
https://mdb.link/mdb200
https://mdb.link/mdb300

Topics we cover

How do Indexes work

When to use Indexes

How MongoDB uses them

Index types and Options

Performance and limitations

Compression

What are Indexes for

Speed up queries and update/sort operations

Avoid disk I/O

Reduce overall computation

The aim is to reduce the resources required for an operation.
Indexes enable us to dramatically change the resources we use, i.e., memory or disk.
The disk should be avoided where possible as it is slower than the memory.
Indexes are sorted so no extra computation required when querying results in sorted order.

Index misconceptions

Misconception	Reality
MongoDB is so fast that it doesn't need indexes	
Every field is automatically indexed	
NoSQL uses hashes, not indexes	

Index misconceptions

Misconception	Reality
MongoDB is so fast that it doesn't need indexes	Incorrect or missing indexes are the main cause of performance issues
Every field is automatically indexed	Indexes must be manually created. The choice and order of the indexed fields is essential for performance
NoSQL uses hashes, not indexes	MongoDB stores indexes in a B-Tree data structure with index keys in sorted order

B-Tree indexes have been used by database systems since the 1970s,

How do Indexes work

Data in an index is ordered so it can be searched efficiently.

Values in an index point to **document identity**.

As a result, if the document moves the index doesn't change.

Data in an index is ordered.

- When looking for specific information, you don't need to look in every document.
- Indexes are in a binary (BTree) structure, so O(log N) to lookup.
- Indexes are traversed by value and then point to a document.

Values in a MongoDB index point to document **identity**.

- Identity is a 64-bit value in a hidden field assigned when a document is inserted.
- Documents in collections are in BTrees ordered by **identity**.
- It's like an internal primary key.
- It's not maintained between replicas though
  - You can see them by adding .showRecordId() to a cursor
  - If their physical location changes, the index stays the same

Index Prefix Compression

Index keys (sorted)

```
applaud
apple
application
ball
banana
```

Index entries (prefix compression)

```
(0,"applaud"),
(4,"e"),
(4,"ication"),
(0,"ball"),
(2,"nana")
```


Because index keys are sorted, we can take advantage of the fact that they may share some prefix. MongoDB will then store each index entry as a delta from the previous one. For identical keys, there just needs to be a pointer to the previous identical entry.

When entries no longer share a prefix, the compression chain starts again.

Smaller indexes reduce I/O requirements and optimise cache usage.

When to use an index

Every query should use an index!

Without an index, MongoDB must scan the entire collection:

For each document:

1. Read from disk
2. Decompress into memory
3. Store in cache

Collection scans are expensive and kill the cache

If you run a collection scan, especially on a large collection, MongoDB might need to evict data cached from previous queries. So those queries will have to start from scratch next time they are run.

A collection scan performs poorly and can cause other queries to perform poorly too.

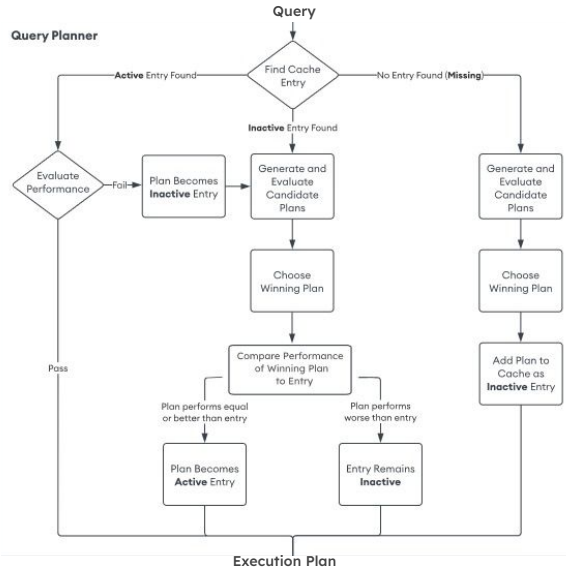
MongoDB can be configured to disallow queries without indexes - see https://www.mongodb.com/docs/manual/reference/parameters/#mongodb-parameter-param.notablescan

Using and choosing Indexes

MongoDB checks in the **PlanCache** to see if an optimal index has been chosen before.

If not:

- Picks all candidate indexes
- Runs query using them to score which is most efficient
- Adds its choice of best index to the PlanCache



The plan cache is a repository that contains the access plans for queries that were optimized
MongoDB uses an empirical technique to look for the best index, this does not require any collection statistics and is well suited
to the simple - single collection type queries mongoDB performs, normally with just one index.

If there are no appropriate indexes then it will run a collection scan.
If there is only one it will use that.
If there are more than one it could use it will 'Race' them - the index which returns 100 results first is chosen as the winner.

Using and choosing Indexes

Plan cache entries are automatically evicted when:

- Using that index becomes less efficient
- A new relevant index is added
- The server is restarted

Can also be manually cleared using:

- db.<collectionName>.getPlanCache().clear()

Query explain results show information on the **Query Plan** and **execution statistics**

MongoDB keeps a cache of the best index for any given "Shape" of query and how well it performed in the "Race" and if future queries do badly it will have a new "Race"
If a new, relevant index is created, or the server is restarted, MongoDB will invalidate the cache entry and try again.
We can run commands to see what's in the plan cache if we have a complex issue to debug but normally there will only be one useful index.

Viewing Explain Plans

explain() shows how MongoDB will execute the query:

- Winning Plan
- Collection Scan or Index Scan
- Optionally execution statistics as well

```
db.runCommand(
   {
     explain: {
        find: "products",
        query: { quantity: { $gt:
 50 } }
        },
     verbosity: "queryPlanner"
   }
)
```

```
db.products.explain("queryPlanner").find(
{quantity:{$gt: 50}}
)
```


*Note: Mongosh does not support explain against deleteOne or deleteMany commands, but you can still run an explain against a deletion using the runCommand syntax, for example:

```
> db.runCommand({explain: {delete:
"listingsAndReviews_big_collection", deletes: [{q:{ property_type:
"Palace" }, limit:1}]}, verbosity: "queryPlanner"})
```

An explain on an update() does not actually modify the document(s).

See https://www.mongodb.com/docs/manual/reference/command/explain/

Explain verbosity

`queryPlanner` shows the winning query plan but does not execute query.

`executionStats` executes query and gathers statistics.

`allPlansExecution` runs all candidate plans and gathers statistics.

If you do not specify a verbosity - the default is **`"queryPlanner"`**

By default, we only see what the DB Engine intends to do

“executionStats” gives us more detail, such as how long the query takes to run and how much data it has to examine

“allPlansExecution” runs all candidate plans and gathers statistics for comparison

Interpret Explain Plan Results

We can see this query looked at all 5,555 documents, returning 11 in 10 milliseconds

We can create an index to improve this.

Key metrics are in green here.

Execution stages include:

- COLLSCAN
- IXSCAN
- FETCH

```
MongoDB> use sample_airbnb
switched to db sample_airbnb

MongoDB> db.listingsAndReviews.find({
            number_of_reviews:50
         }).explain("executionStats")

 ...
      executionStats: {
            executionSuccess: true,
            nReturned: 11,
            executionTimeMillis: 10,
            totalKeysExamined: 0,
            totalDocsExamined: 5555,
            executionStages: {
            stage: 'filter',
            planNodeId: 1,
            nReturned: 11,
            executionTimeMillisEstimate: 10,

 ...
```


nReturned is how many documents this stage returns - e.g., the index may narrow to 100 documents, but then an unindexed filter drops that to 10 documents
totalKeysExamined - number of index entries
totalDocsExamined - number of documents read

Ideally, all three of the above are the same number.

Stage shows us whether a collection scan or index was used.

Applying explain()

A flag is sent to the server with the operation to say it's an explain command.

If the function does not return a cursor, the explain flag needs to be set in the collection object instead before calling the function

We can set this on the cursor as we do for sort() or limit()

Example: In a count() or update(), we set the flag on the collection object we call it with.


```
peoplecoll = db.people
explainpeoplecoll = peoplecoll.explain()
explainpeoplecoll.count()
```

A query is sent to the server once we start requesting from the cursor - so we set a flag on the cursor to request the explain plan rather than the results.
If we don't have a cursor, calling explain() on a collection returns a collection with that flag set.

A collection scan

explain() on cursor shows the query mechanics

COLLSCAN: Collection Scan

Every document in the collection looked at

Very inefficient

```
MongoDB> use sample_airbnb
switched to db sample_airbnb

MongoDB> db.listingsAndReviews.find({
                   number_of_reviews: 50 }).explain()
{
  explainVersion: '2',
  queryPlanner: {
    namespace: 'sample_airbnb.listingsAndReviews',
    indexFilterSet: false,
    parsedQuery: {
      number_of_reviews: {
        '$eq': 50
      }
    },
  ...
    winningPlan: {
      queryPlan: {
        stage: 'COLLSCAN',
        planNodeId: 1,
        filter: {
          number_of_reviews: {
            '$eq': 50
          }
        },
        direction: 'forward'
      },
```


Queries tend to always be quick on small datasets, so we will use tools to see the implications rather than merely observing time.

The example query has no index, so the DB Engine must look through every document in the collection.

If the collection is too big and isn't cached in RAM, then a huge amount of Disk IO will be consumed, and it will be very slow.

Even if in RAM, it is a lot of data to look through

Creating a simple index

In development, we can create an index using `createIndex()` on the primary

In production, we need to look at the impact this will have.

There are better ways to do it in production.

```
MongoDB> db.listingsAndReviews.createIndex({
            number_of_reviews:1
         })

number_of_reviews_1
```


Creating an index takes an object of fields with an ascending index (1) or descending index (-1) option.

In languages where members of objects aren't ordered, the syntax is a little different.

The return value is the name of the newly created index.

Building indexes in production system needs to consider the impact on the server, cache, disks, and any locking that may occur.

Building indexes is an administrative task and should be performed during maintenance windows, or outside of write-heavy load times.

See https://www.mongodb.com/docs/manual/core/index-creation/

Index Demonstration

Here we see two stages:

IXSCAN (Look through the index) returning a list of 11 documents identities

FETCH (Read known document) getting the document itself

The total time was one millisecond

```
MongoDB> db.listingsAndReviews.find({
      number_of_reviews:50}).explain("executionStats")
...
    winningPlan: {
      queryPlan: {
        stage: 'FETCH',
        planNodeId: 2,
        inputStage: {
          stage: 'IXSCAN',
          planNodeId: 1,
          keyPattern: {
            number_of_reviews: 1
          },
          indexName: 'number_of_reviews_1',
...
      executionStats: {
            nReturned: 11,
            executionTimeMillis: 1,
            totalKeysExamined: 11,
            totalDocsExamined: 11,
...
```


The example shows improved efficiency of using an index - Note how **totalKeysExamined**, **totalDocsExamined**, and **nReturned** are all the same.
Same behavior as an RDBMS

```
 executionStats: {
  executionSuccess: true,
  nReturned: 11,
  executionTimeMillis: 1,
  totalKeysExamined: 11,
  totalDocsExamined: 11,
  executionStages: {
   stage: 'nlj',
   planNodeId: 2,
   nReturned: 11,
   executionTimeMillisEstimate: 0,
   opens: 1,
   closes: 1,
   saveState: 0,
   restoreState: 0,
   isEOF: 1,
   totalDocsExamined: 11,
   totalKeysExamined: 11,
   collectionScans: 0,
   collectionSeeks: 11,
   indexScans: 0,
   indexSeeks: 1,
   indexesUsed: [
 ...
```

Index types

Single-field indexes

Compound indexes

Multikey indexes

Geospatial indexes

Other Index types:
Text indexes, Hashed indexes, Wildcard indexes

Index types are similar to other DB vendors.
Text indexes, hashed indexes and wildcard indexes are note recommended for general use

- Text - rather use Atlas Search if possible
- Hashed - B-Tree structure is optimized for consecutive values, but hashes are semi-random
- Wildcard - only useful in very specific areas, but can become very large

Single-Field Indexes

Optimize finding values for a given field

- Specified as field name and direction
- The direction is irrelevant for a single field index
- The field itself can be any data type.

Avoid indexing an Object type:

- Indexes the whole object as a comparable blob
- Can use it for range searches, but better to index individual fields as less error prone

Indexing an Array is covered later

A single-field index is the most basic kind
Builds a tree of all values pointing to the documents they are in
It makes looking up a value O(log n) versus O(n)
Allows range queries and sorting to be performant too
When querying objects, both exact matches and ranges rely on the **field order** being the same. We must check our app programming language does not rearrange the field order.

Unique Index

Indexes can enforce a unique constraint

Example,

```
db.a.createIndex({custid: 1}, {unique: true})
```

NULL is a value and so only one record can have a NULL in unique field.

NB: Recall that a missing field in a document has a logical value of 'null'.

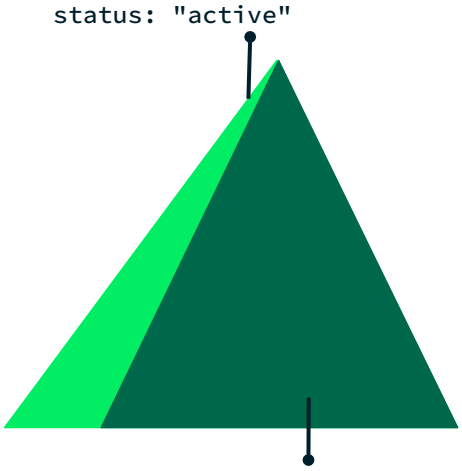
Partial Index

Partial indexes only index documents in a collection that meet specific *filter criteria*

Benefits:

- Smaller index size
- Lower write overhead

```
db.users.createIndex(
    {"email": 1},
    {partialFilterExpression:
        {status: "active"} })
```



We can use a partial index to only index documents where an expression is true.

This reduces the size of the index. More of it can fit in the cache so I/O calls are reduced. Targeted queries will benefit from better read performance as less index data needs to be read.

For documents that don't match the filter, no index entry is created. Writes to your "archived" data will not incur the additional cost of updating the index.

Hashed Indexes

Hashed Indexes index a 20 byte md5 of the BSON value

Support exact match only

Cannot be used for unique constraints

Can potentially reduce index size if original values are large

Downside: random values in a BTree use excessive resources

```
db.people.createIndex({ name : "hashed" })
```


Hashed indexing creates performance challenges for range matches etc., so it should be used with caution.
Random values (Hashes or traditional GUIDS) in a BTree maximize the requirement for RAM and Disk/IO and so should be avoided. This is covered later in the course.

Exercise - Indexing

Use the **sample_airbnb** database and the **listingsAndReviews** collection:

Find the name of the host with the most total listings (this is an existing field)

Create an index to support the query

Calculate how much more efficient it is now with this index

Note - this is a tricky question for a number of reasons!

A simpler version of this could also be provided if the above is too hard for the audience:
Find the name of the hosts that have listings greater than 1000 (instead of most total listings)

Listing Indexes

`getIndexes()` on a collection gives us the index definitions for that collection.

```
MongoDB> db.listingsAndReviews.getIndexes()

[
  { v: 2, key: { _id: 1 }, name: '_id_' },

  ...

  {
      v: 2,
      key: { number_of_reviews: 1 },
      name: 'number_of_reviews_1'
  }
]
```


getIndexes() shows index information (the number of indexes may vary from what you see here on the slide)

Index sizes

We can call `stats()` method and look at the **indexSizes** key to see how large each index is in bytes.

```
MongoDB> db.listingsAndReviews.stats().indexSizes

{
  _id_: 143360,
  property_type_1_room_type_1_beds_1: 65536,
  name_1: 253952,
  'address.location_2dsphere': 98304,
  number_of_reviews_1: 45056
}
```


Among other options, the scaling factor can be passed to see stats in desired unit. **`db.listingsAndReviews.stats({ scale: 1024 }).indexSizes`** shows the index sizes in KB.

Indexes and Performance

Indexes improve read performance when used

Each index adds **~10%** overhead for writes (Hashed indexes, multikey indexes, text indexes, and wildcard indexes can add more)

An index is modified any time a document:

- Is inserted (applies to most indexes)
- Is deleted (applies to most indexes)
- Is updated in such a way that its indexed fields change

Indexes must be applied with careful consideration as they do create overhead when writing data
Unused indexes should be identified and removed
At times the partial indexes remain unaffected while insert or delete operations.

Index Limitations

Up to 64 indexes per collection - Avoid being close to this upper bound

Write performance degrades to unusable between 20 and 30

4 indexes per collection is a good recommended number

The hard limit is 64 indexes per collection, but you should not have anywhere near this number

Use Indexes with Care

Every query should use an index

Every index should be used by a query

Indexes require server memory, be mindful about the choice of key

Depending on the size and available resources, indexes will either be used from disk or cached. You should aim to fit indexes in the cache. Otherwise, performance will be seriously impacted.

Quiz Time!

#1. Which 3 operations can explain be used on?

A	find()	B	aggregate()	C	sort()
D	$expr	E	delete()		

Answer in the next slide.

#1. What operations can explain be used on?

A find()

B aggregate()

C sort()

D $expr

E delete()

explain() can be applied to finds, aggregations, deletes, updates and count.

$expr is a specific update operator not the operation itself

sort() is a modifier for find, not a distinct operation. However, you can (and should!) explain find operations where you sort the results.

#2. Select the correct statement about MongoDB Indexes:

A	use prefix compression	B	are stored as hash tables	C	point to the disk location of a document
D	must be held entirely in RAM	E	should be created up-front		

#2. Select the correct statement about MongoDB Indexes:

A	use prefix compression	B	are stored as hash tables	C	point to the disk location of a document
D	must be held entirely in RAM	E	should be created up-front		

MongoDB Indexes use "prefix compression" to keep their size low. They are stored internally as B-Trees (specifically B+ Trees in WiredTiger) and contain the unique recordId of the documents they point to. Indexes can be partially paged into the cache as required.

Unlike RDBMSs, MongoDB indexes should be created according to the needs of your queries. Identify the queries you use most and then build the indexes that would optimise their performance.

Recap

Indexes improve efficiency and speed of reads

Every query should use an index

MongoDB explain() shows how indexes are being chosen and used in query executions

Exercise Answers

Answer - Exercise: Indexing

Find the name of the host with the most total listings

```
> db.listingsAndReviews.find({},{"host.host_total_listings_count":1,
"host.host_name":1}).sort({"host.host_total_listings_count":-1}).limit(1)
{ "_id" : "12902610", "host" : { "host_name" : "Sonder", "host_total_listings_count" : 1198 } }
```

Create an index to support the query and show how much more efficient it is:

```
"executionTimeMillis" : 11,
"totalDocsExamined" : 5555,
"works" : 5559,

> db.listingsAndReviews.createIndex({"host.host_total_listings_count": 1})
"executionTimeMillis" : 1,
"totalKeysExamined" : 1,
"totalDocsExamined" : 1,
"works" : 2,
```


Note: In the first query where we apply a sort and limit, MongoDB doesn't guarantee consistent return of the same document especially when there are simultaneous writes happening. It is advisable to include the _id field in sort if consistency is required.